

SPOTIFY MUSIC ANALYTICS

Delivery: End-to-End Data Analytics Portfolio Project

Solution Design & Technical Documentation

Data Modeling, Exploratory Analysis, and SQL Business Intelligence for a 114,000-Track Spotify Dataset

Version 1.0
July 2026

Prepared by: Srilakshmi N P
github.com/reeseekat

Table of Contents

Table of Contents	2
Executive Summary	5
1. Project Background	5
1.1 Project Overview	5
1.2 Business Context	5
1.3 Analytical Process	5
1.4 Problem Statement	6
2. Project Requirements & Objectives	6
2.1 Project Objective	6
2.2 Business Questions Answered	6
Python EDA	6
SQL Business Intelligence	6
2.3 Non-Functional Considerations	7
2.4 Success Criteria	7
3. Solution Summary	7
3.1 Solution Summary	7
3.2 Solution Objectives	8
3.3 High-Level Solution Architecture	8
3.4 Key Features of the Solution	8
3.5 Business Benefits (Portfolio Framing)	9
Chapter 4 – Solution Architecture & Technical Design	9
4.1 Purpose	9
4.2 High-Level Architecture	9
4.3 Solution Components	9
4.3.1 Data Source Layer	9
4.3.2 Data Preparation Layer	10
4.3.3 Data Modeling / Normalization Layer	10
4.3.4 Business Logic Layer (SQL)	10
4.3.5 Presentation Layer	10
4.4 Technical Design Principles	10
4.5 Technical Design Decisions	10
4.6 Benefits of the Architecture	11
Chapter 5 – Data Model Design	11
5.1 Purpose	11
5.2 Data Modeling Approach	11
5.3 Data Model Components	11
Fact Table	11

Dimension Tables	12
Bridge Table.....	12
5.4 Fact Table — fact_tracks	12
5.5 Dimension Tables	12
dim_genre	12
dim_artist	12
5.6 Bridge Table — bridge_track_artist	12
5.7 Relationship Design.....	12
5.8 Data Quality Note — Non-Unique track_id	12
5.9 Design Decisions.....	13
5.10 Chapter Summary.....	13
Chapter 6 – Data Transformation (Python / pandas)	13
6.1 Purpose.....	13
6.2 Transformation Overview	13
6.3 Design Consideration — Genre Normalization	14
6.4 Design Consideration — Artist Normalization	14
6.5 Data Quality Measures.....	14
6.6 ETL Design Principles	14
6.7 Chapter Summary.....	15
Chapter 7 – Exploratory Data Analysis (Python)	15
7.1 Purpose.....	15
7.2 EDA Highlights	15
1. Popularity Distribution	15
2. Top Genres by Popularity.....	16
3. Danceability vs. Popularity.....	16
4. Explicit vs. Non-Explicit Popularity.....	17
5. Energy vs. Popularity	17
6. Happiest Genres by Valence	18
7. Audio Feature Correlation Heatmap	18
8. Track Duration vs. Popularity	19
7.3 Key EDA Findings	19
9. Acoustic vs. Energy Extremes by Genre	20
Chapter 8 – SQL Business Intelligence Analysis	21
8.1 Purpose.....	21
8.2 SQL Calculation Architecture	21
8.3 Query Catalog.....	21
8.4 Chapter 1 — Artist Performance Analysis.....	22
Business Question 4 — Top artist per genre (RANK() window function)	22

Business Question 2 — Consistency (STDDEV)	22
8.5 Chapter 2 — Collaboration Analysis	22
Business Question 7 — Top collaborator pairs (self-join)	22
Business Questions 5 & 6 — Collaboration share and popularity impact	23
8.6 Chapter 3 — Genre Benchmarking	23
Business Question 8 — Genre vs. overall average (correlated subquery)	23
Business Question 9 — Explicit content over-indexing	23
Business Question 10 — Genre concentration	23
8.7 Chapter 4 — Catalog Analysis	24
Business Question 11 — Album depth vs. popularity	24
Business Question 12 — "Hidden gems" (multi-CTE composition)	24
8.8 Reading This Chapter Alongside the SQL File	24
8.9 SQL Design Principles	24
8.10 Chapter Summary	25
Chapter 9 – Reproducibility & Maintenance	25
9.1 Purpose	25
9.2 Project File Structure	25
9.3 Reproducing the Analysis	25
9.4 Maintenance Guidelines	26
9.5 Troubleshooting Guide	26
9.6 Chapter Summary	26
Chapter 10 – Solution Evaluation & Future Roadmap	26
10.1 Purpose	26
10.2 Solution Evaluation	26
10.3 Current Limitations	27
10.4 Scalability Assessment	27
10.5 Future Enhancement Roadmap	27
De-duplicated Track Grain	27
Additional Data Sources	27
Dashboard Layer	27
Automated Refresh & Testing	27
10.6 Lessons Learned	28
10.7 Recommendations	28
10.8 Chapter Summary	28

Executive Summary

This project analyzes 114,000 Spotify tracks spanning 114 genres to understand what drives music popularity on the platform, and to demonstrate an end-to-end analytics workflow: raw data ingestion, cleaning, relational (star schema) modeling, exploratory data analysis in Python, and business-question-driven analysis in SQL.

The raw dataset arrived as a single flat CSV file, with genre and artist information repeated across rows in a way that violates normal form and makes relational analysis difficult. The project normalizes this into a proper star schema — a central fact table of tracks joined to genre and artist dimensions, connected through a many-to-many bridge table — then layers two distinct analytical passes on top of it: exploratory data analysis in Python (distributions, correlations, audio-feature relationships) and business intelligence analysis in SQL (artist performance, collaboration patterns, genre benchmarking, and catalog structure).

The result is a fully reproducible portfolio project that mirrors the structure of a real analytics engagement: a documented data model, a cleaning and transformation pipeline, an EDA notebook, and a catalogue of 12 SQL business questions answered with intermediate-to-advanced SQL techniques (CTEs, window functions, self-joins, correlated subqueries, and CASE-based segmentation).

Key outcomes include a validated relational data model, quantified findings on genre and artist performance, a documented data-quality issue in the source dataset (duplicate track identifiers across genre tags) that was identified and accounted for during analysis, and a clear foundation that can be extended with additional data sources, automated refresh, or a front-end dashboard as the project evolves.

This document records the project background, data model design, transformation logic, SQL business logic, analytical findings, and a roadmap for future enhancement.

1. Project Background

1.1 Project Overview

This is a self-directed portfolio project built to demonstrate the complete analytics lifecycle on a realistic, large-scale dataset — from messy raw data to a documented relational model and a business-question-driven analytical layer. It uses the public Spotify Tracks Dataset (114,000 tracks across 114 genres) as source data and is intended to be read as a working example of how a real analytics team would approach the same dataset for a music streaming business.

1.2 Business Context

Music streaming platforms operate in an environment where thousands of new tracks are added continuously, and only a small fraction ever reach meaningful listener engagement. Understanding what separates a popular track from an undiscovered one — genre, explicit content, energy, duration, collaboration patterns, artist consistency — is directly useful to content strategy, curation, and A&R (artist and repertoire) decision-making. This project treats the Spotify dataset as if it were an internal content-analytics data warehouse being queried by a Business Intelligence analyst.

1.3 Analytical Process

The project follows a structured, two-stage analytical process:

- Stage 1 — Exploratory Data Analysis (Python): cleaning, normalization, and open-ended exploration of distributions, correlations, and audio-feature relationships.
- Stage 2 — Business Intelligence Analysis (SQL): a curated set of business questions run directly against the normalized relational schema, organized into four analytical chapters (Artist Performance, Collaboration, Genre Benchmarking, Catalog Analysis).

This separation mirrors how many analytics teams operate in practice: Python/pandas for flexible, iterative exploration and cleaning, and SQL for repeatable, auditable business reporting against a stable relational model.

1.4 Problem Statement

The source dataset, as delivered, is a single flat table with 21 columns where genre is represented as a repeated string per track and artists are stored as a semicolon-delimited list within a single field. This structure:

- Violates First Normal Form (multiple artists packed into one field).
- Makes genre-level and artist-level aggregation error-prone without repeated string parsing.
- Provides no clean way to represent the many-to-many relationship between tracks and artists.
- Is not queryable with standard relational JOIN logic without prior transformation.

The project's first objective was therefore to transform this flat structure into a proper star schema before any relational (SQL) analysis could begin.

2. Project Requirements & Objectives

2.1 Project Objective

The primary objective was to build a fully normalized, queryable data model from a single flat CSV file, and to use that model to answer a structured set of business questions using both Python (exploratory) and SQL (business intelligence) techniques — producing a portfolio artifact that demonstrates data modeling, ETL/cleaning, EDA, and intermediate-to-advanced SQL in combination.

2.2 Business Questions Answered

Python EDA

- Which genres dominate Spotify in terms of popularity?
- Does explicit content affect track performance?
- What audio features define a popular track?
- Which genres make listeners feel the most positive (valence)?
- Do high-energy tracks perform better than calm ones?
- What is the optimal track duration for mainstream success?
- How do audio features correlate with each other and with popularity?

SQL Business Intelligence

- Which artists perform consistently well versus rely on a single hit?
- Do collaborations outperform solo tracks?
- Who are an artist's most frequent, best-performing collaborators?

- How does each genre benchmark against the platform-wide average?
- Which genres over-index on explicit content?
- Which genres are dominated by a few artists versus wide open?
- Which low-visibility artists are producing breakout "hidden gem" tracks?

2.3 Non-Functional Considerations

ID	Consideration
NFR-01	The data model must be understandable to someone reading the SQL cold, without needing the Python notebook open.
NFR-02	Every SQL query must be independently executable against the delivered schema.
NFR-03	The transformation pipeline must be reproducible from the original flat CSV.
NFR-04	Known data-quality issues in the source data must be documented, not silently absorbed.
NFR-05	The project must use only the columns and tables that genuinely exist in the delivered schema — no invented fields.

2.4 Success Criteria

Success Criteria	Measurement
Schema normalized correctly	Star schema loads cleanly into MySQL / DuckDB with all foreign-key relationships intact.
EDA complete	All seven Python EDA business questions answered with supporting charts.
SQL BI layer complete	12 SQL business questions delivered across 4 chapters, each executable and inference-backed.
Findings are evidence-based	Every inference in the SQL notebook is derived from actually running the query, not assumed.
Documentation complete	This document accurately reflects the delivered schema, pipeline, and SQL layer.

3. Solution Summary

3.1 Solution Summary

The delivered solution consolidates a single flat Spotify export into a normalized star schema, consisting of one fact table (fact_tracks) and two dimension tables (dim_genre, dim_artist) connected through a many-to-many bridge table (bridge_track_artist). The transformation was performed in a Python/pandas notebook, and the resulting tables were loaded into MySQL for relational analysis.

Analysis was then performed in two layers: an exploratory layer in Python covering distributions, correlations, and audio-feature relationships; and a business intelligence layer in SQL covering artist performance, collaboration patterns, genre benchmarking, and catalog structure — questions that specifically require the relational model rather than a flat dataframe.

3.2 Solution Objectives

- Normalize a flat, denormalized export into a proper relational star schema.
- Separate exploratory (Python) analysis from repeatable business-question analysis (SQL).
- Demonstrate a coherent SQL analytical story — not a disconnected list of queries — progressing from simple aggregation to window functions, self-joins, and multi-CTE analysis.
- Document and account for real-world data-quality issues discovered during analysis, rather than ignoring them.
- Produce documentation suitable for a data analytics portfolio.

3.3 High-Level Solution Architecture

Step	Layer	Description
1	Raw Data	Flat Spotify export (spotify_tracks.csv) — 114,000 rows, 21 columns, one row per track.
2	Data Preparation	Python / pandas: drop index artifact column, validate types, check for missing values.
3	Normalization	Genre extracted into dim_genre; artist string exploded into dim_artist + bridge_track_artist; fact_tracks stripped of denormalized fields.
4	Star Schema	fact_tracks (fact) ↔ dim_genre, dim_artist (dimensions) via genre_id and the bridge table.
5	EDA (Python)	Distribution analysis, correlation heatmap, genre/valence/energy/duration analysis, visualized with matplotlib/seaborn.
6	SQL BI Layer	12 business questions across 4 chapters, executed in MySQL 8 using JOINS, CTEs, window functions, subqueries, and CASE logic.

3.4 Key Features of the Solution

Feature	Description
Star Schema Modeling	Converts a flat, denormalized export into a fact/dimension/bridge structure supporting scalable relational joins.
Artist Performance Analysis	Ranks and segments artists by consistency, catalog depth, and per-genre leadership using window functions.
Collaboration Analysis	Quantifies how often artists collaborate and whether collaboration correlates with higher popularity, using a self-join on the bridge table.
Genre Benchmarking	Compares every genre against the platform-wide average using correlated subqueries and CTEs.
Catalog Analysis	Surfaces catalog-depth patterns and "hidden gem" tracks using multi-CTE composition.
Data Quality Auditing	Documents a genuine data-quality issue (non-unique track_id) discovered while executing the SQL layer, and shows how it affects specific results.

3.5 Business Benefits (Portfolio Framing)

While this is a portfolio project rather than a live production system, it is designed to mirror the value a similar solution would provide inside a real streaming or label organization:

- Replaces ad-hoc, one-off analysis with a stable, queryable relational model.
- Demonstrates the ability to identify and communicate data-quality risk rather than silently producing misleading numbers.
- Shows a coherent analytical narrative — each SQL chapter builds on the last — rather than a disconnected exercise list.
- Provides a template data model and SQL pattern library that could be extended to a real streaming analytics use case with minimal rework.

Chapter 4 – Solution Architecture & Technical Design

4.1 Purpose

This chapter describes the technical architecture of the project: how data flows from the raw CSV export into a normalized relational model, and how that model is analyzed by two independent toolchains (Python and SQL). The intent is to let a reader unfamiliar with the project understand the pipeline end-to-end without needing to read every line of code.

4.2 High-Level Architecture

The solution follows a layered architecture separating raw ingestion, cleaning, modeling, and analysis into independent, sequential stages. This keeps each stage independently re-runnable — the SQL layer, for example, can be re-executed against the exported CSVs without touching the Python notebook at all.

Stage	Tool	Output
Raw Source Data	spotify_tracks.csv	114,000 rows, 21 columns, flat structure.
Data Preparation	pandas	Cleaned dataframe, index artifact removed, types validated.
Normalization	pandas	genre_df, artist_bridge, fact_df exported as separate CSVs.
Relational Load	MySQL 8 / DuckDB	fact_tracks, dim_genre, dim_artist, bridge_track_artist tables.
Exploratory Analysis	pandas, matplotlib, seaborn	spotify_eda.ipynb — charts and written inferences.
Business Intelligence Analysis	SQL (MySQL 8 dialect)	sql_analysis.sql (foundation) + sql_analysis_bi.sql (12 business questions).

4.3 Solution Components

4.3.1 Data Source Layer

The source data is a single Excel/CSV-style export (spotify_tracks.csv) containing 114,000 rows. Each row represents one track tagged with a single genre — meaning the same physical track can appear

multiple times if it is associated with more than one genre. This characteristic of the source data is carried through the entire pipeline and is discussed in detail in Chapter 5.

4.3.2 Data Preparation Layer

pandas is used for cleaning and validation: dropping the non-informative Unnamed: 0 index column inherited from the CSV export, confirming that audio-feature columns are correctly typed as floats, confirming explicit is boolean, and checking for missing values.

4.3.3 Data Modeling / Normalization Layer

The flat dataframe is split into a star schema: a genre dimension is extracted from the repeated track_genre string column, an artist dimension and bridge table are built by splitting and exploding the semicolon-delimited artists column, and the fact table is stripped of both denormalized fields once the corresponding foreign keys and bridge relationships exist.

4.3.4 Business Logic Layer (SQL)

All business calculations — quantity comparisons, percentage benchmarks, segmentation, ranking — are implemented directly in SQL against the normalized schema, using CTEs and window functions rather than being pre-computed in pandas. This keeps the business logic auditable and reproducible by anyone with access to the four CSV exports and a SQL engine.

4.3.5 Presentation Layer

Findings are presented in two forms: Python-generated charts embedded directly in the EDA notebook (matplotlib/seaborn), and written, data-backed inferences embedded as SQL comments directly beneath each business-intelligence query in sql_analysis_bi.sql.

4.4 Technical Design Principles

Principle	Implementation
Separation of concerns	Exploratory analysis (Python) is kept separate from repeatable business reporting (SQL).
Reproducibility	Every SQL query is independently executable against the delivered CSV exports / MySQL tables.
Auditability	Inferences are derived from actually executing each query, not inferred from memory.
Progressive complexity	SQL business questions are ordered from simple aggregation to advanced multi-CTE composition within each chapter.
Minimal surface area	Only columns and tables that genuinely exist in the delivered schema are used — no invented fields.

4.5 Technical Design Decisions

Decision	Reason
Star schema over flat table	Enables genre- and artist-level aggregation without repeated string parsing, and supports the many-to-many track/artist relationship correctly.
Separate bridge table for artists	A track can have multiple artists and an artist can have multiple tracks; a bridge table is the only structure that models this correctly.

Decision	Reason
SQL business logic kept out of Python	Keeps the business-intelligence layer independently auditable and portable to any SQL engine, not tied to a specific pandas pipeline.
DuckDB used for validation	Allows every SQL query to be executed directly against the raw CSV exports for verification, without requiring a running MySQL server.
Data-quality issues documented, not silently fixed	A discovered issue (non-unique track_id) is called out explicitly in both the SQL file and this document rather than being quietly patched, preserving analytical transparency.

4.6 Benefits of the Architecture

- Clear separation between exploratory and business-reporting analysis.
- Every SQL business question is independently reproducible and testable.
- The relational model can be re-loaded into any SQL engine (MySQL, PostgreSQL, DuckDB) with minimal changes.
- Data-quality issues are surfaced rather than hidden, improving the trustworthiness of the analysis.
- The layered design provides a template that scales cleanly if additional data sources (e.g. streaming counts, playlist data) were added later.

Chapter 5 – Data Model Design

5.1 Purpose

This chapter describes the logical data model developed for the project: how the datasets are organized, the relationships between them, and the design decisions — including a real data-quality issue discovered during analysis — that shape how results should be interpreted.

5.2 Data Modeling Approach

The project uses a Star Schema, the standard dimensional modeling approach for analytical workloads. Transactional/measurable data (tracks and their audio features) is separated from descriptive data (genres, artists), which:

- Improves query performance and simplifies JOIN logic.
- Enables consistent filtering across genre- and artist-level analysis.
- Supports reusable, auditable SQL business logic.
- Matches the structure a real streaming-analytics warehouse would use.

5.3 Data Model Components

Fact Table

Table	Description	Row Count
fact_tracks	Track-level audio features and popularity metrics, one row per track-genre tag	114,000

Dimension Tables

Table	Description	Row Count
dim_genre	114 unique genres	114
dim_artist	Unique artist master list	29,859

Bridge Table

Table	Description	Row Count
bridge_track_artist	Many-to-many relationship between tracks and artists	158,293

5.4 Fact Table — fact_tracks

Primary attributes:

- track_id, album_name, track_name
- popularity, duration_ms, explicit
- danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, time_signature
- genre_id (foreign key to dim_genre)

5.5 Dimension Tables

dim_genre

genre_id (surrogate key), genre_name — extracted from the repeated track_genre string column in the source export.

dim_artist

artist_id (surrogate key), artist_name — extracted by splitting the semicolon-delimited artists field and exploding it into individual rows.

5.6 Bridge Table — bridge_track_artist

track_id, artist_id — models the many-to-many relationship between tracks and artists. This structure is what makes collaboration analysis (Chapter 8) possible: without it, a track credited to two artists would need to be represented as a single delimited string, which cannot be joined against an artist dimension.

5.7 Relationship Design

From → To	Cardinality
fact_tracks → dim_genre	Many-to-One (via genre_id)
bridge_track_artist → fact_tracks	Many-to-One (via track_id)
bridge_track_artist → dim_artist	Many-to-One (via artist_id)

5.8 Data Quality Note — Non-Unique track_id

This issue was not anticipated during modeling — it surfaced only once the SQL business-intelligence layer was actually executed against the real data (Chapter 8, Business Question 2), when an artist-

consistency query returned a suspicious result: several artists showed a popularity standard deviation of exactly zero. That result was the symptom; the cause turned out to be a structural characteristic of the source export itself.

⚠ What's actually going on with track_id

A track that Spotify tags with more than one genre is exported as more than one row — same track_id, same audio features, different genre_id. Out of 114,000 rows, only about 89,741 represent a genuinely distinct track; the rest are genre-repeat rows.

This is a property of the delivered dataset, not something the normalization step introduced or should silently paper over.

Practically, it means any downstream count that groups by track_id or joins through it — how many tracks an artist has, how many times two artists appear together, how many tracks sit in an album — is really counting genre-tagged rows, not physical songs.

Every place in Chapter 8 where this changes how a number should be read is flagged at the point it happens, rather than asking the reader to remember this note while working through twelve separate queries.

5.9 Design Decisions

Decision	Reason
Surrogate keys for genre_id / artist_id	Avoids relying on inconsistent free-text strings as join keys.
Bridge table instead of a delimited artist string	The only structure that correctly represents a many-to-many relationship in a relational model.
track_id retained as the fact table's natural key despite non-uniqueness	Preserves fidelity to the source data rather than silently deduplicating and losing genre-tag information; the trade-off is documented rather than hidden.

5.10 Chapter Summary

The project is built on a star schema consisting of one fact table, two dimension tables, and one bridge table. The model supports genre- and artist-level analysis directly through SQL joins, and a real data-quality characteristic of the source data (non-unique track_id) has been identified, documented, and accounted for in the analysis that follows.

Chapter 6 – Data Transformation (Python / pandas)

6.1 Purpose

This chapter documents the transformation pipeline implemented in spotify_eda.ipynb that converts the flat source export into the normalized star schema described in Chapter 5.

6.2 Transformation Overview

Step	Operation	Purpose
1	Load spotify_tracks.csv	Bring in the raw 114,000-row flat export.
2	Drop Unnamed: 0 column	Removes a non-informative CSV index artifact.

Step	Operation	Purpose
3	Validate data types	Confirm audio features are floats and explicit is boolean.
4	Extract unique genres → dim_genre	Assigns a surrogate genre_id to each unique genre string.
5	Merge genre_id into fact table, drop track_genre	Replaces the repeated text column with a foreign key.
6	Split & explode artists column	Converts the semicolon-delimited artist string into one row per (track, artist) pair.
7	Build dim_artist	Extracts the unique artist list with a surrogate artist_id.
8	Build bridge_track_artist	Stores the (track_id, artist_id) pairs from the exploded artist list.
9	Drop artists / artist_list from fact table	Removes denormalized artist data, enforcing 1NF on the fact table.
10	Check & drop duplicate track_id rows during export	Identified during data cleaning; full duplicates were removed, though the underlying genre-repetition characteristic remains (see Chapter 5.8).
11	Export fact_tracks.csv, dim_genre.csv, dim_artist.csv, bridge_track_artist.csv	Produces the four CSVs used to load the relational model.

6.3 Design Consideration — Genre Normalization

The track_genre column in the source data contains repeating categorical values. Reducing this to a dedicated dim_genre table with a surrogate genre_id avoids storing the same genre string thousands of times and enables clean relational joins for genre-level aggregation.

6.4 Design Consideration — Artist Normalization

The artists column stores multiple artists per track separated by a semicolon, representing a many-to-many relationship: one track can have multiple artists, and one artist can appear on multiple tracks. Storing this as a delimited string violates First Normal Form and cannot be joined relationally. Splitting and exploding the column into a bridge table structure (bridge_track_artist) is the standard relational solution and is what enables the collaboration analysis in Chapter 8.

6.5 Data Quality Measures

- Verified audio-feature columns are numeric (float) and explicit is boolean.
- Confirmed negligible missing values across the dataset.
- Checked for and removed fully duplicate track_id rows during the final export step.
- Confirmed the resulting CSV row counts against the original source file before loading into SQL.

6.6 ETL Design Principles

Principle	Implementation
Minimal transformation	Only the transformations required for normalization and cleanliness were applied; audio-feature values were left untouched.

Principle	Implementation
Traceability	Each transformation step is documented inline in the notebook with a written rationale.
Consistency	Surrogate keys (genre_id, artist_id) are generated once and reused consistently across all exported tables.
Reproducibility	The pipeline runs end-to-end from the single source CSV to the four normalized exports.

6.7 Chapter Summary

The transformation pipeline converts a single flat, denormalized Spotify export into a clean star schema through genre extraction, artist explosion into a bridge table, and standard data-quality validation. The following chapter builds on this foundation with the exploratory analysis performed in Python.

Chapter 7 – Exploratory Data Analysis (Python)

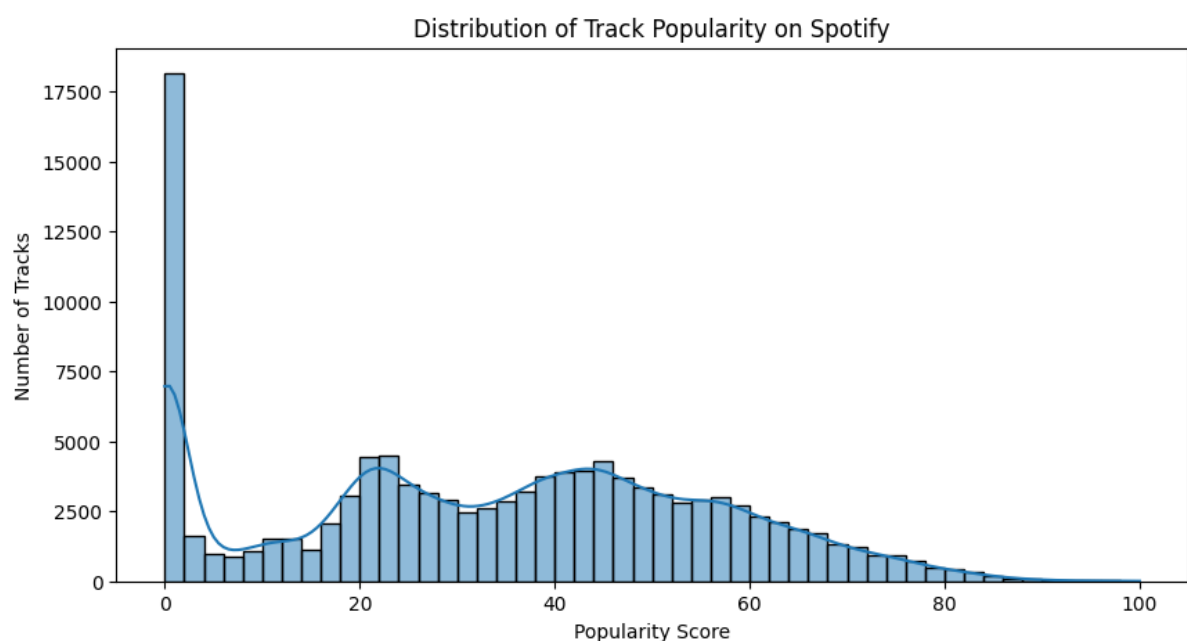
7.1 Purpose

This chapter summarizes the exploratory analysis performed in `spotify_eda.ipynb` once the star schema was in place. The goal of this stage was open-ended: understand distributions, relationships between audio features, and general genre-level patterns before moving to the structured business-question analysis in SQL (Chapter 8).

7.2 EDA Highlights

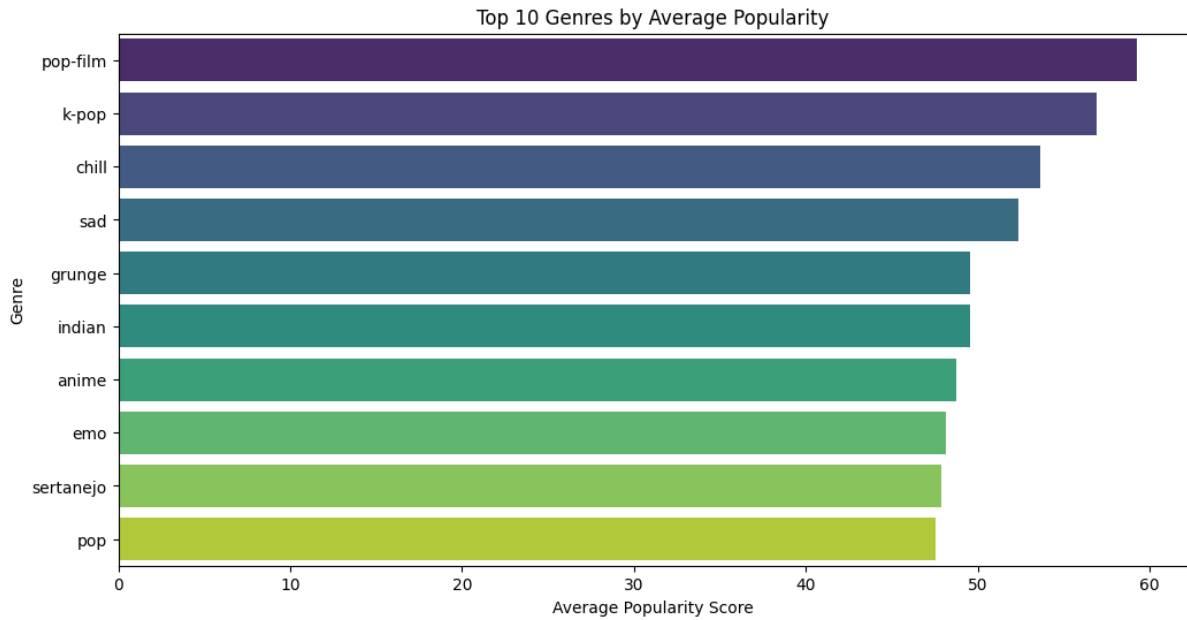
The charts below are the actual outputs saved inside `spotify_eda.ipynb` — not regenerated for this document. Each is followed by a one-line reading, not a re-explanation; the notebook has the full narrative and code for each.

1. Popularity Distribution



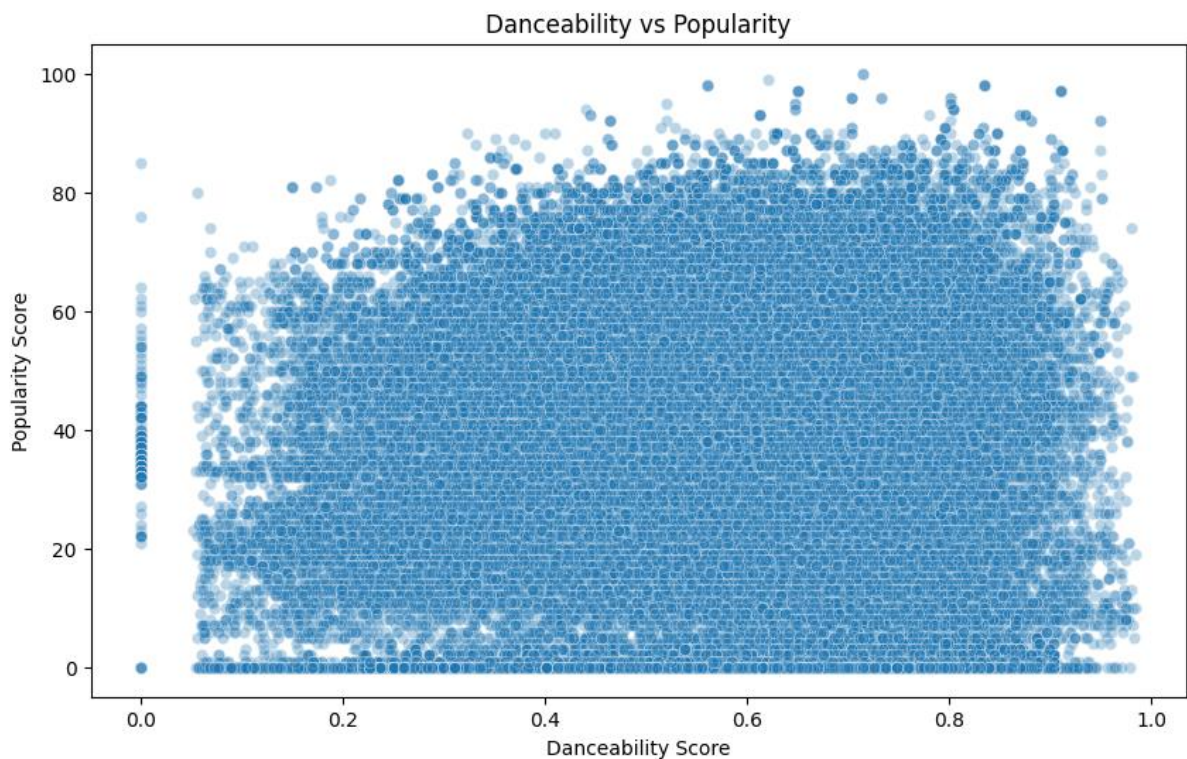
75% of tracks score below 50/100 — most content on the platform goes largely undiscovered.

2. Top Genres by Popularity



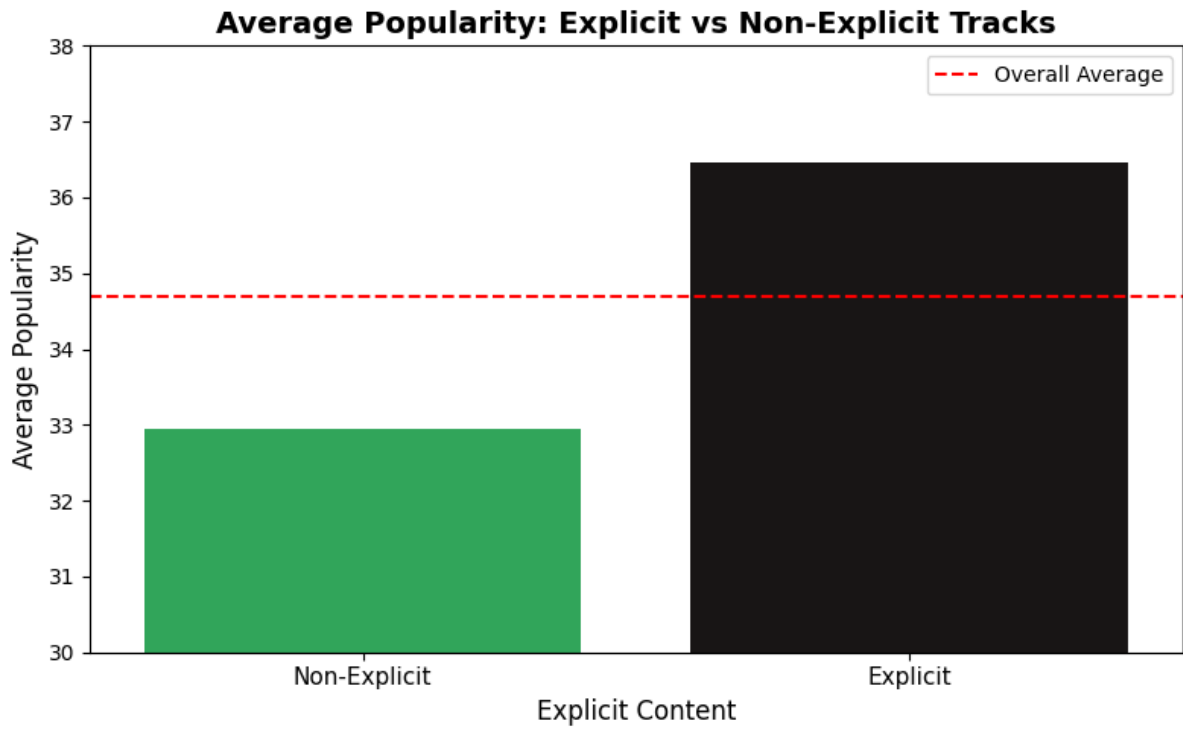
pop-film and k-pop lead — cultural relevance outweighs traditional genre preference.

3. Danceability vs. Popularity



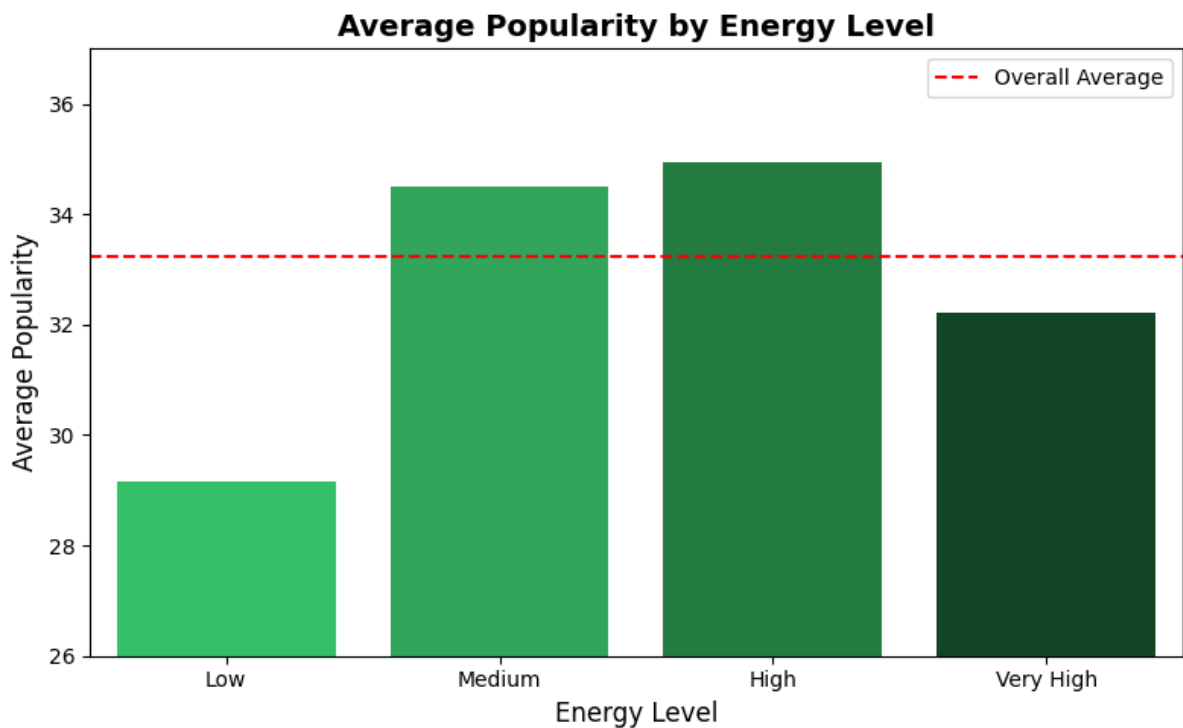
A weak positive trend — danceability nudges popularity up but doesn't drive it.

4. Explicit vs. Non-Explicit Popularity



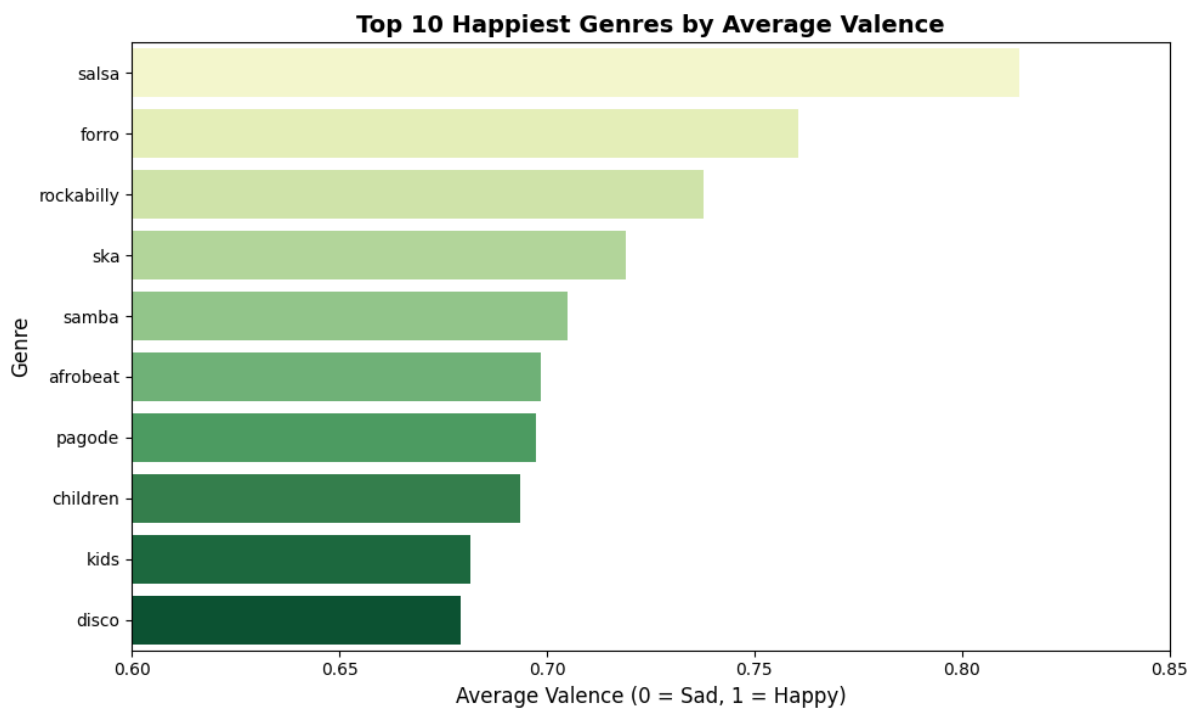
Explicit tracks average higher popularity — the discovery algorithm doesn't penalize explicit content.

5. Energy vs. Popularity



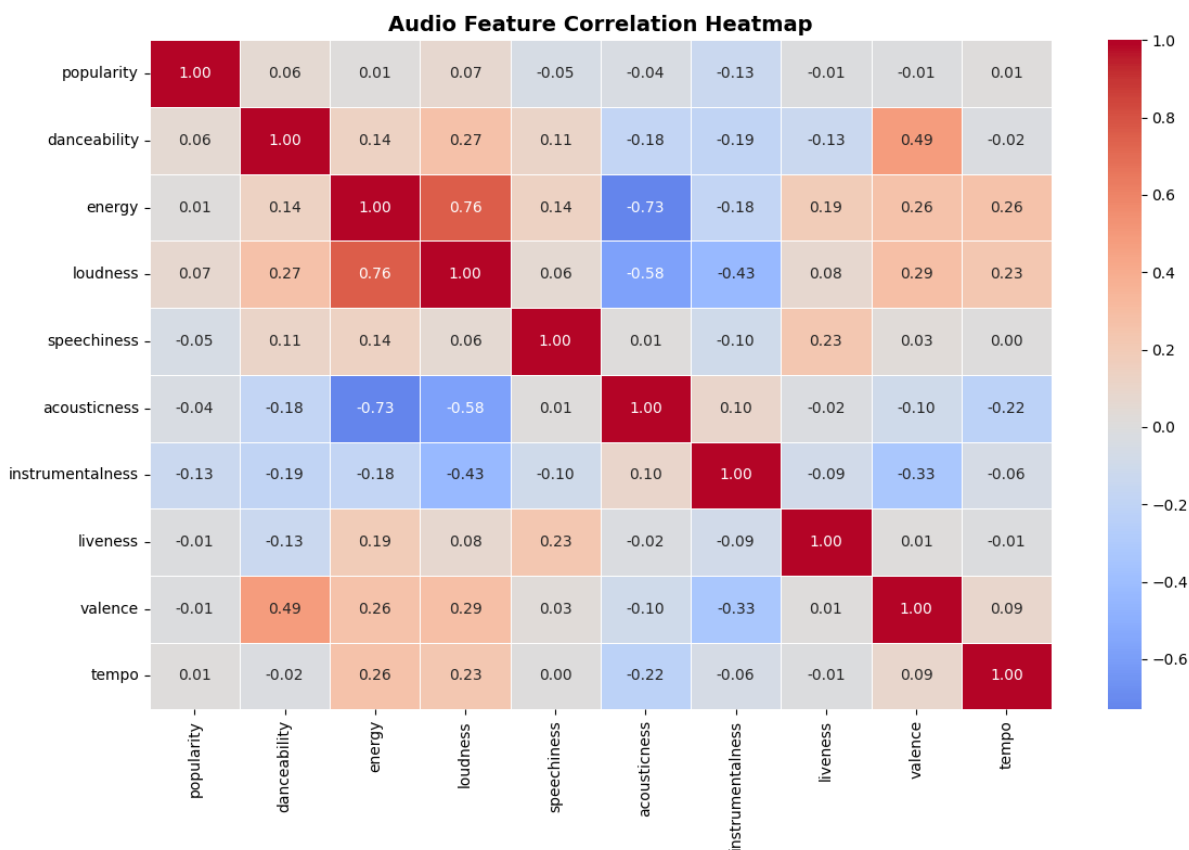
Popularity peaks at moderate-to-high energy; the very-high-energy bucket falls back off.

6. Happiest Genres by Valence



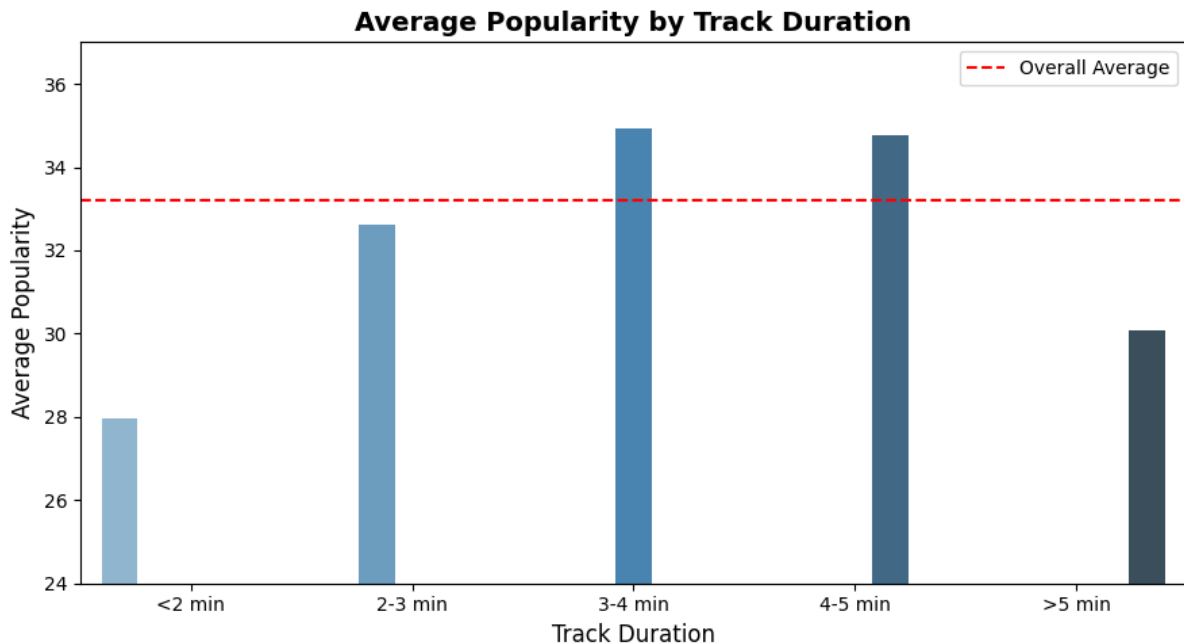
Salsa, forro, and similar dance-rooted genres lead on emotional positivity.

7. Audio Feature Correlation Heatmap



No single feature strongly predicts popularity; energy and acousticness are near-opposite.

8. Track Duration vs. Popularity



3–5 minute tracks consistently outperform; very short and very long tracks both underperform.

7.3 Key EDA Findings

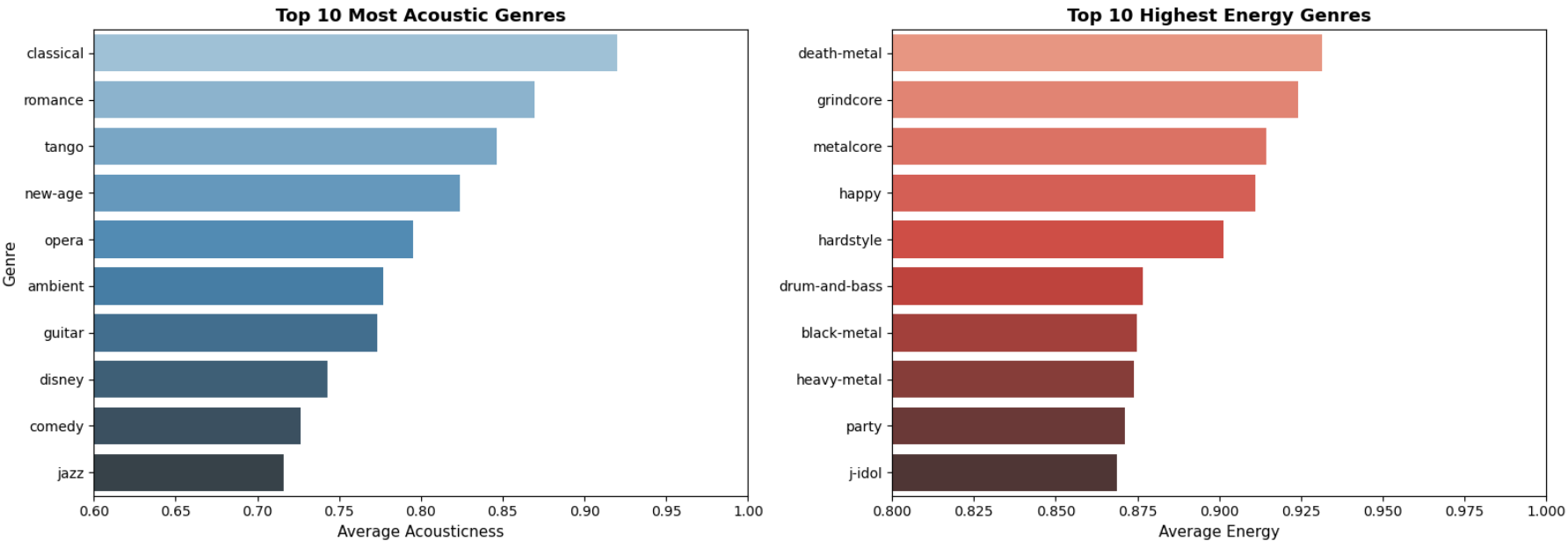
- K-pop and pop-film dominate popularity — culturally-driven genres outperform traditional ones.
- Explicit tracks slightly outperform, likely driven by hip-hop and rap dominance in the explicit category.
- No single audio feature predicts popularity — it is a multi-dimensional outcome.
- 3–5 minutes is the optimal track length for mainstream success.
- Acoustic and energy represent two largely separate listener experiences on the platform.

These findings deliberately are not re-derived in the SQL layer that follows. Chapter 8 focuses instead on questions the relational model is uniquely positioned to answer — artist- and collaboration-level patterns that require joining across the fact, dimension, and bridge tables.

One chart from this section — the acoustic/energy genre comparison — is a wide, two-panel figure that is easier to read in landscape. It's reproduced at full size on the following page.

9. Acoustic vs. Energy Extremes by Genre

Acoustic vs Energy: The Two Extremes of Music



Classical/romance dominate acoustic; death-metal/grindcore dominate energy — largely separate genre clusters.

Chapter 8 – SQL Business Intelligence Analysis

8.1 Purpose

This chapter documents the SQL business-intelligence layer built directly on the star schema (`sql_analysis_bi.sql`). It deliberately avoids repeating the audio-feature EDA from Chapter 7 and instead focuses on questions the relational model is uniquely suited to answer: artist performance, collaboration patterns, genre benchmarking, and catalog structure. Every query in this chapter was executed against the actual data (via DuckDB, validated against the same CSV exports used to load MySQL), and every inference below reflects real output, not an assumption.

8.2 SQL Calculation Architecture

The 12 business questions are organized into four chapters that build on each other, moving from simple aggregation to advanced multi-table composition:

Chapter	Focus	SQL Techniques Introduced
1. Artist Performance	Who performs well, and how consistently?	JOIN + GROUP BY, HAVING, STDDEV, CASE segmentation, RANK() window function
2. Collaboration	Do artists working together perform differently?	Bridge-table aggregation, self-join with a de-duplication predicate
3. Genre Benchmarking	How does each genre compare to the platform?	Correlated subquery, multi-CTE comparison, CROSS JOIN
4. Catalog Analysis	What does catalog structure reveal?	GROUP BY + HAVING at album grain, stacked CTEs with window functions

8.3 Query Catalog

#	Chapter	Business Question
1	Artist Performance	Top 10 artists by average popularity (min. 5 tracks)
2	Artist Performance	Most consistent prolific artists (STDDEV of popularity)
3	Artist Performance	One-Hit Wonder / Consistent Performer / Underperformer segmentation
4	Artist Performance	Top-ranked artist within each genre (RANK() window function)
5	Collaboration	Solo vs. collaboration share of the catalog
6	Collaboration	Popularity comparison: collaboration vs. solo tracks
7	Collaboration	Most frequent, best-performing collaborator pairs (self-join)
8	Genre Benchmarking	Genre average popularity vs. platform-wide average
9	Genre Benchmarking	Genres that over-index on explicit content
10	Genre Benchmarking	Tracks-per-artist ratio (genre concentration vs. fragmentation)
11	Catalog Analysis	Album depth vs. average popularity

#	Chapter	Business Question
12	Catalog Analysis	"Hidden gem" tracks — above-genre-average popularity from low-visibility artists

8.4 Chapter 1 — Artist Performance Analysis

This chapter identifies which artists perform well, and — more importantly — which perform reliably well versus being carried by a single hit.

Business Question 4 — Top artist per genre (RANK() window function)

```
RANK() OVER (
  PARTITION BY d.genre_name
  ORDER BY AVG(f.popularity) DESC
) AS genre_rank
```

Partitioning the ranking by genre — rather than computing one global leaderboard — surfaces the top artist inside every genre simultaneously. This is the first window function used in the notebook and the pattern that Business Question 12 later builds on.

The output is worth reading for what it says about the schema, not just about artists. The same artist can legitimately lead several "different" genre leaderboards at once, which is a direct consequence of how granular and overlapping this dataset's genre taxonomy is (see the full figures in `sql_analysis_bi.sql`). For a real content-curation team, that would be a prompt to ask whether the genre taxonomy itself needs consolidating before genre-level leaderboards are surfaced to end users.

Business Question 2 — Consistency (STDDEV)

This is the query that first exposed the `track_id` issue described in Chapter 5.8 — not through inspection of the schema, but because the output looked wrong. An artist-consistency ranking should show low but non-zero variation for a genuinely reliable artist; seeing it collapse to precisely zero for several artists in a row was the tell that something upstream was duplicating rows rather than describing genuine performance stability. It is a useful reminder that validating against real output catches problems that reasoning about a query in the abstract would not.

8.5 Chapter 2 — Collaboration Analysis

This chapter uses the bridge table to answer questions a flat dataframe cannot: how common is collaboration, does it correlate with popularity, and who works together most often?

Business Question 7 — Top collaborator pairs (self-join)

```
FROM bridge_track_artist b1
INNER JOIN bridge_track_artist b2
  ON b1.track_id = b2.track_id
  AND b1.artist_id < b2.artist_id
```

The self-join on `bridge_track_artist` is the most technically advanced single line in the notebook: joining the bridge table to itself on a shared `track_id` surfaces every pair of co-credited artists, and the `b1.artist_id < b2.artist_id` predicate is what prevents each pair from being counted twice (once in each order).

This is also the query where the `track_id` issue bites hardest, because a self-join multiplies the effect of any row duplication rather than just carrying it forward. The most frequent pairs surfaced (concentrated in reggaeton and Latin music) are directionally believable, but the raw counts

attached to them should not be quoted as a literal number of songs — the full figures and the caveat are in `sql_analysis_bi.sql`. The general lesson generalizes beyond this project: a self-join is a powerful pattern for finding co-occurrence, but it is also the pattern most sensitive to a non-unique join key.

Business Questions 5 & 6 — Collaboration share and popularity impact

Metric	Result
Share of catalog that is collaborative	38.11% (34,200 of 89,741 distinct tracks)
Solo track average popularity	32.17
Collaboration average popularity	34.25

The size of the gap between the two averages matters as much as its direction. A roughly two-point difference is real and consistent, but it is not large enough to justify "add a featured artist" as a standalone strategy — it is one input among many, well behind whatever is driving the 20+ point spreads seen in the genre-benchmarking chapter that follows.

8.6 Chapter 3 — Genre Benchmarking

This chapter compares individual genres against the platform-wide average, using two different SQL patterns that express the same underlying comparison.

Business Question 8 — Genre vs. overall average (correlated subquery)

```
SELECT d.genre_name, ROUND(AVG(f.popularity), 2) AS genre_avg_popularity,
       (SELECT ROUND(AVG(popularity), 2) FROM fact_tracks) AS overall_avg_popularity,
       ROUND(AVG(f.popularity) -
             (SELECT AVG(popularity) FROM fact_tracks), 2) AS variance_from_overall_avg
FROM fact_tracks f
INNER JOIN dim_genre d ON f.genre_id = d.genre_id
GROUP BY d.genre_name
```

The shape of the result is the interesting part: the genres sitting below the platform average fall further below it than the genres sitting above it climb above. In practical terms, a handful of niche, low-visibility genres are pulling the platform-wide number down harder than the breakout genres are pulling it up — which suggests that fixing platform-average engagement is more a matter of investing in the long tail than chasing the next pop-film or k-pop moment. The exact per-genre figures are in `sql_analysis_bi.sql`; this section is about what the shape of the distribution implies, not the individual numbers.

Business Question 9 — Explicit content over-indexing

The genre that over-indexes hardest on explicit content is not the one most people would guess. That result is a useful check on assumption-driven analysis: if the query had simply confirmed the obvious stereotype, it would be tempting to skip validating it against the real numbers. Because it didn't, it's a small demonstration of why this project insists on running every query rather than reasoning about expected results from memory.

Business Question 10 — Genre concentration

The tracks-per-artist ratio effectively measures how "open" a genre is on the platform: a high ratio means a small number of established artists account for most of the catalog, while a low ratio means the genre is spread thin across many contributors, each with only a track or two. That distinction matters differently depending on who's asking — a label scouting for under-served genres would look for the fragmented end of this list, while a curator trying to build a reliable genre playlist would look for the concentrated end.

8.7 Chapter 4 — Catalog Analysis

This chapter closes the analytical story by examining catalog structure at the album grain, and then combining genre benchmarking (Chapter 3) with artist performance (Chapter 1) in the most advanced query in the notebook.

Business Question 11 — Album depth vs. popularity

This query was written expecting to find a positive relationship — the intuition being that a deep, well-supported album would tend to have stronger individual tracks. The data says the opposite: the deepest entries in the catalog are almost entirely seasonal or mood compilation playlists mislabeled as albums, and they perform poorly. That reversal is a good example of why this project treats "album" as whatever label the source data assigns rather than assuming it means a curated artist release — the label and the reality can disagree.

Business Question 12 — "Hidden gems" (multi-CTE composition)

```
WITH genre_avg AS (
  SELECT genre_id, AVG(popularity) AS genre_avg_popularity
  FROM fact_tracks GROUP BY genre_id
), artist_catalog_size AS (
  SELECT b.track_id, b.artist_id,
  COUNT(*) OVER (PARTITION BY b.artist_id) AS artist_total_tracks
  FROM bridge_track_artist b
)
SELECT f.track_name, a.artist_name, d.genre_name, f.popularity, ...
WHERE f.popularity > g.genre_avg_popularity + 15
AND acs.artist_total_tracks <= 3
```

This query stacks two independent CTEs — a genre benchmark and a per-artist catalog-size window function — then joins both against the fact table with two simultaneous conditions. The two CTEs should be read as independent building blocks rather than a single top-to-bottom narrative, and it's the query where every technique from the earlier chapters (benchmarking, window functions, and the track_id caution from Chapter 5.8) has to be held in mind at once.

The practical output — a shortlist of tracks that outperform their genre despite coming from an artist with a thin catalog — is exactly the kind of list a playlist-discovery team would want as a starting point, precisely because it's derived from comparison rather than raw popularity alone. The full result set is listed in `sql_analysis_bi.sql`.

8.8 Reading This Chapter Alongside the SQL File

Every number cited above is drawn from actually executing the corresponding query, and the complete set of inferences — including several smaller findings not singled out here — lives as a comment directly beneath each query in `sql_analysis_bi.sql`. This chapter is deliberately selective: it picks out the handful of results that changed how a query, a table, or a design decision should be understood, rather than re-listing everything the SQL file already says. If a specific figure is needed for citation, the SQL file is the source to quote from directly.

8.9 SQL Design Principles

- Every query executes against only the columns and tables that genuinely exist in the delivered schema.
- Business questions progress from simple aggregation to window functions, self-joins, and multi-CTE composition within each chapter.

- Every inference is derived from actually running the query against the real data, not inferred from memory.
- A genuine data-quality issue (non-unique `track_id`) is documented once at the schema level and referenced specifically wherever it materially changes how a result should be read.

8.10 Chapter Summary

The SQL business-intelligence layer answers 12 business questions across four coherent chapters, demonstrating JOINS, GROUP BY/HAVING, CASE-based segmentation, window functions, self-joins, correlated subqueries, and stacked CTEs — all executed against real data, with a documented and consistently-applied caveat about a genuine data-quality issue in the source dataset.

Chapter 9 – Reproducibility & Maintenance

9.1 Purpose

This chapter describes how to reproduce the project end-to-end, how the source files relate to one another, and how to extend the analysis with new data. Since this is a portfolio project rather than a live production system, this replaces the deployment/operations chapter of a client-facing document with a practical reproducibility guide.

9.2 Project File Structure

File	Purpose
spotify_tracks.csv	Original flat source export (114,000 rows).
spotify_eda.ipynb	Python notebook: cleaning, normalization, and exploratory analysis.
fact_tracks.csv	Normalized fact table export.
dim_genre.csv	Genre dimension export.
dim_artist.csv	Artist dimension export.
bridge_track_artist.csv	Bridge table export.
sql_analysis.sql	Database/table creation (DDL) and initial foundational queries.
sql_analysis_bi.sql	12-query SQL business intelligence layer, organized into 4 chapters.
README.md	Project overview, key findings, and file guide.

9.3 Reproducing the Analysis

1. Run `spotify_eda.ipynb` top to bottom against `spotify_tracks.csv` to regenerate the four normalized CSV exports.
2. Run `sql_analysis.sql` against a MySQL 8 instance to create the schema and load the exported CSVs.
3. Run `sql_analysis_bi.sql` against the same schema to reproduce all 12 business-intelligence queries.
4. Alternatively, the SQL business-intelligence layer can be executed directly against the four CSV exports using DuckDB, without requiring a running MySQL server — this is how every inference in Chapter 8 was validated.

9.4 Maintenance Guidelines

Activity	Recommended Trigger
Re-run the EDA notebook	If the source dataset changes or is refreshed.
Re-validate SQL inferences	After any change to fact_tracks, dim_genre, dim_artist, or bridge_track_artist.
Re-check the data-quality note (Chapter 5.8)	If the source dataset is replaced — the non-unique track_id characteristic may not hold for a different export.
Update README.md	Whenever chapters or key findings are added to the SQL business-intelligence layer.

9.5 Troubleshooting Guide

Issue	Likely Cause	Recommended Action
Foreign key errors loading fact_tracks	dim_genre / dim_artist not loaded first	Load dimension tables before the fact and bridge tables.
Unexpectedly high artist track counts	Non-unique track_id (see Chapter 5.8)	De-duplicate on track_id before aggregating if a strictly unique-track count is required.
Encoding errors on CSV load	UTF-8 BOM in exported CSVs	Load with UTF-8 (with BOM) encoding, or strip the BOM before import.
"key" column errors in MySQL	key is a reserved word	Wrap the column name in backticks (`key`) in any raw SQL that references it directly.

9.6 Chapter Summary

The project is fully reproducible from the original source CSV using either a MySQL 8 instance or DuckDB directly against the exported CSV files. The known data-quality characteristic of the source data should be re-verified if the underlying dataset is ever replaced or refreshed.

Chapter 10 – Solution Evaluation & Future Roadmap

10.1 Purpose

This chapter evaluates the completed project against its original objectives, documents known limitations, and outlines a roadmap for future enhancement.

10.2 Solution Evaluation

The project successfully converts a flat, denormalized Spotify export into a documented star schema and delivers two complementary layers of analysis on top of it: exploratory analysis in Python and a 12-query business-intelligence layer in SQL. Key strengths include:

- A well-structured star schema that correctly models the many-to-many track/artist relationship via a bridge table.

- A coherent SQL analytical story that progresses from simple aggregation to window functions, self-joins, and multi-CTE composition.
- Every SQL inference is grounded in actual query execution against the real data, not assumption.
- A genuine data-quality issue was discovered, documented, and consistently accounted for rather than silently absorbed.
- The project is reproducible using either MySQL or DuckDB, with no dependency on a specific runtime.

10.3 Current Limitations

- `track_id` is not a true unique key in the source data (see Chapter 5.8), which requires care when interpreting raw counts in several queries.
- The dataset is a static, one-time export; there is no automated refresh pipeline.
- Genre taxonomy in the source data is granular and overlapping (e.g. "latin" vs. "latino" vs. "reggaeton" as separate genres), which can make genre-level comparisons less intuitive than a consolidated taxonomy would allow.
- The project does not include a front-end dashboard; all findings are delivered through the notebook, SQL comments, and this document.

10.4 Scalability Assessment

The star schema and SQL business-logic pattern generalize cleanly to a larger or continuously-refreshed dataset. Because business logic is expressed in SQL against stable table structures rather than hard-coded in the notebook, additional business questions can be added to `sql_analysis_bi.sql` without modifying the underlying schema, provided the new questions can be answered from the existing fact/dimension/bridge tables.

10.5 Future Enhancement Roadmap

De-duplicated Track Grain

A cleaned, fully de-duplicated version of `fact_tracks` (one row per physical track, with genres modeled as their own bridge table) would resolve the data-quality caveat raised throughout Chapter 8 and allow exact rather than directional collaboration and consistency counts.

Additional Data Sources

Streaming counts, playlist inclusion data, or release-date metadata would extend the model with genuine time-series and trend analysis, which the current static export does not support.

Dashboard Layer

The SQL business-intelligence layer could be connected to a BI tool (e.g. Power BI or Tableau) to turn the 12 static queries into an interactive, filterable dashboard.

Automated Refresh & Testing

A lightweight test suite validating row counts, foreign-key integrity, and the presence of the known data-quality characteristic would make the pipeline safe to re-run against future data exports without manual re-verification.

10.6 Lessons Learned

Building the star schema before writing any SQL business logic paid off directly — every one of the 12 business-intelligence queries was expressible as a straightforward join against the fact, dimension, and bridge tables, with no need to fall back on string parsing or ad-hoc logic.

Actually executing every query against the real data — rather than writing plausible-sounding SQL and reasoning about expected results from memory — surfaced a genuine data-quality issue (non-unique `track_id`) that would otherwise have silently produced misleading counts in several queries. This reinforced the value of validating analytical output against ground truth rather than trusting query logic alone.

10.7 Recommendations

- Treat the non-unique `track_id` characteristic as a first-class fact about this dataset, not an edge case, in any future extension of the analysis.
- Prioritize a de-duplicated track grain before adding further artist- or collaboration-level metrics.
- Keep SQL business logic and Python exploratory analysis clearly separated as the project grows, to preserve auditability.
- Re-verify all data-quality assumptions if the source dataset is ever refreshed or replaced.

10.8 Chapter Summary

The project meets its original objectives: a documented, normalized data model; a complete exploratory analysis; and a 12-query SQL business-intelligence layer demonstrating intermediate-to-advanced SQL technique, all grounded in real, executed query results. The roadmap above outlines a clear path from portfolio project to a more production-oriented analytics solution.